Divide-and-Conquer: Quicksort

Design and Analysis of Algorithms

Dhannya SM

Quicksort

Disadvantage of mergesort:

- Not in-place

Quicksort

- Uses divide and conquer technique
- Partitions the input list into two sublists
- Partitioning happens based on a **pivot**

Quicksort

- One sublist holds values less than that of the pivot and the other sublist holds values that are greater than the pivot

Quicksort

- One sublist holds values less than that of the pivot and the other sublist holds values that are greater than the pivot

2	6	3	7	12	9	10	11

Can you guess the pivot?

Quicksort

- One sublist holds values less than that of the pivot and the other sublist holds values that are greater than the pivot

2	6	3	7	12	9	10	11

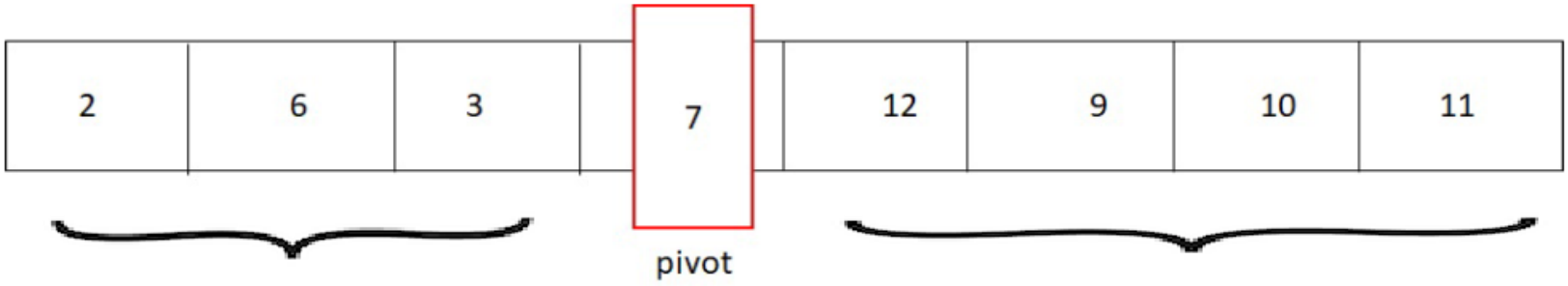
Can you guess the pivot?

2	6	3	7	12	9	10	11

pivot

Quicksort

- Once the pivot is positioned correctly, recursively sort the two sublists

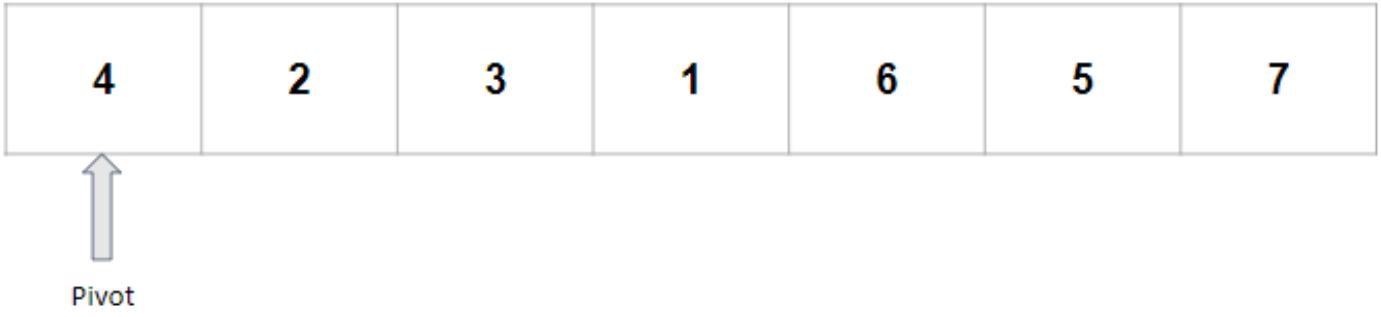


Quicksort Algorithm

- Choose a pivot, here we choose the first element
- Let *low* and *high* be two pointers, where *low* points to the list of values smaller than the pivot and *high* points to the list of values larger than the pivot
- While value in *low* is less than the pivot, move the pointer *low* to the right
- While value in *high* is larger than the pivot, move the pointer *high* to the left
- When *low* has a value larger than the pivot and *high* has a value smaller than the pivot, then exchange the values in *low* and *high*
- Finally, when $low \geq high$, let the index of the element at which they crossed over be i. Exchange element at i and the pivot

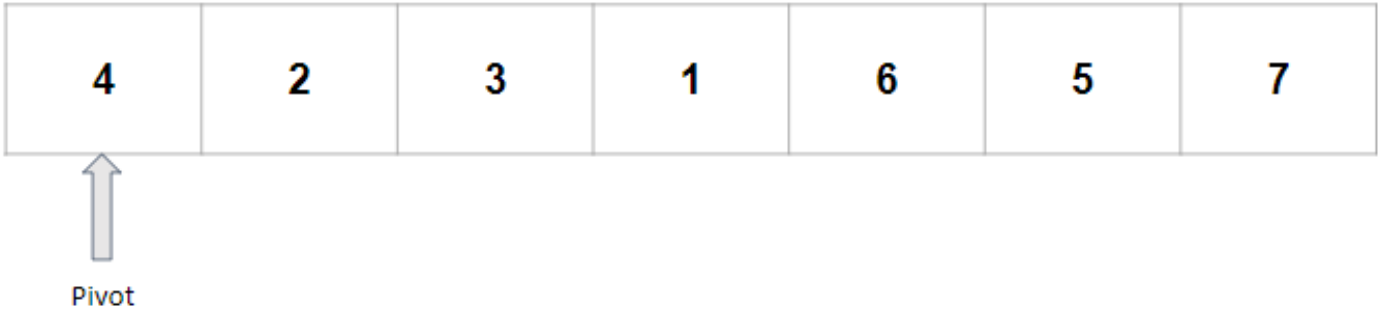
Quicksort Algorithm

- Choose a pivot, here we choose the first element

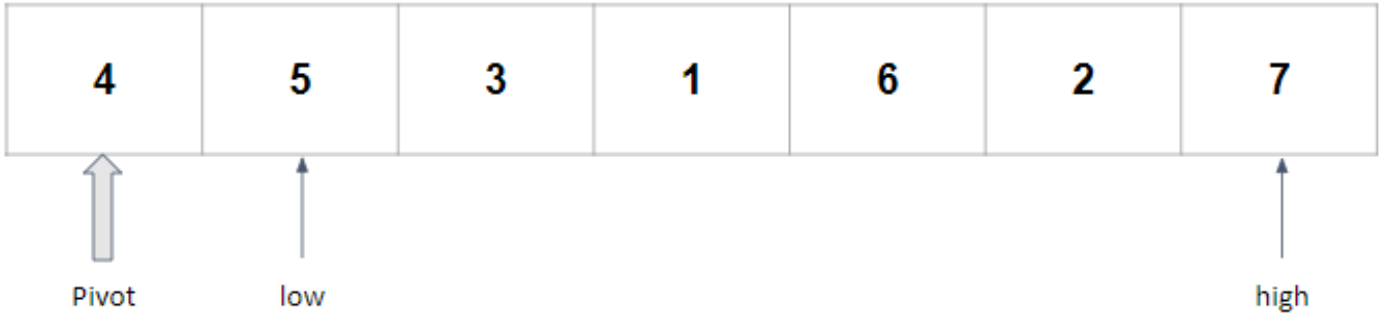


Quicksort Algorithm

- Choose a pivot, here we choose the first element

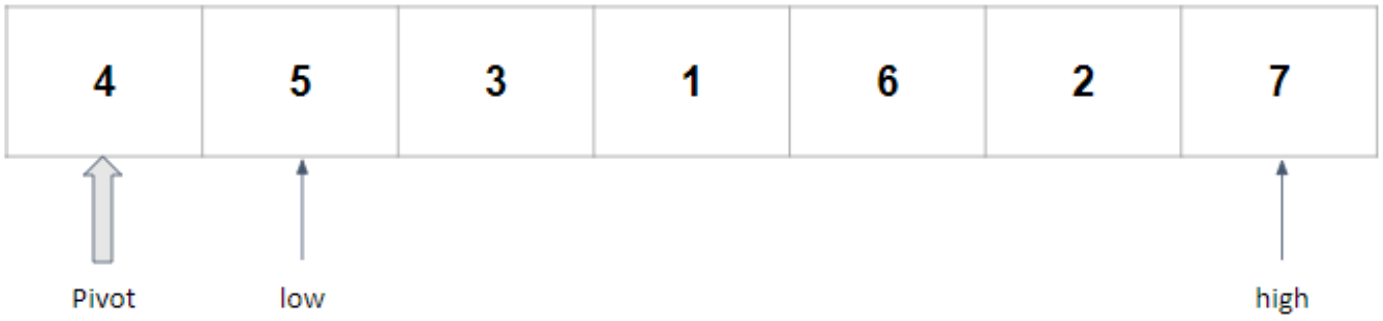


- Let *low* and *high* be two pointers, where *low* points to the list of values smaller than the pivot and *high* points to the list of values larger than the pivot



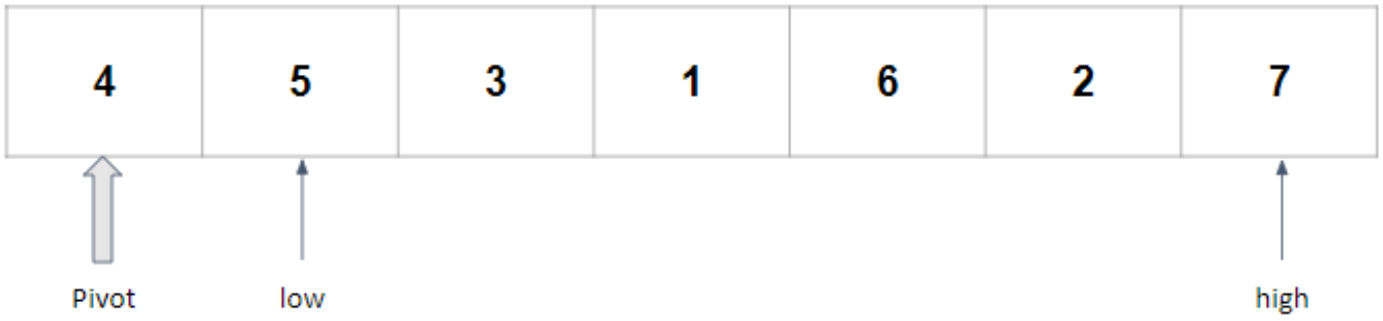
Quicksort Algorithm

- While value in *low* is less than the pivot, move the pointer *low* to the right

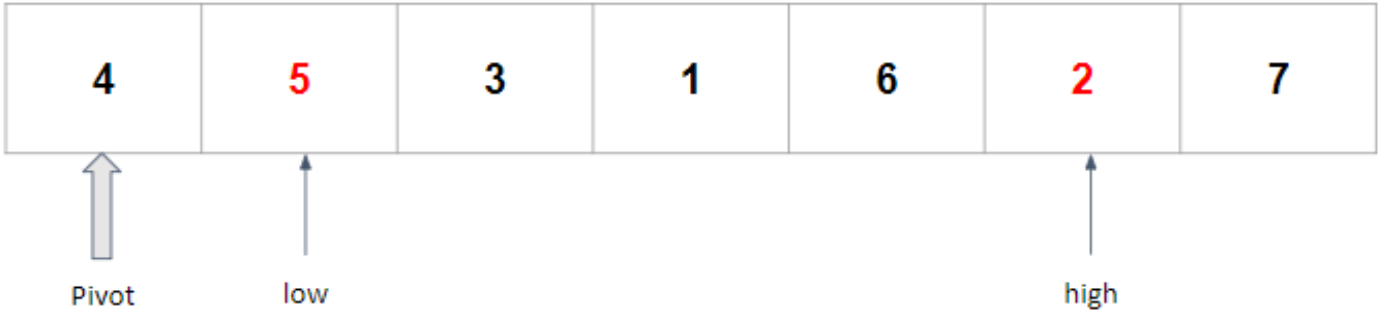


Quicksort Algorithm

- While value in *low* is less than the pivot, move the pointer *low* to the right

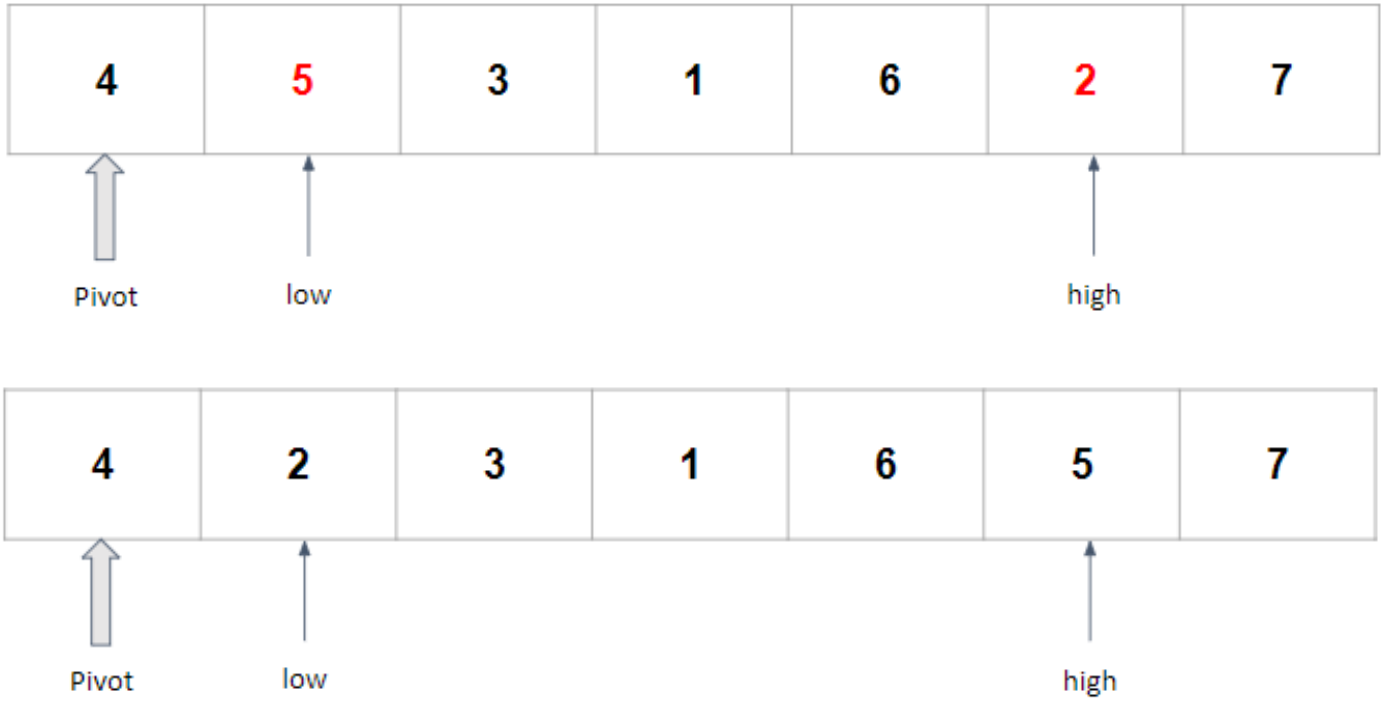


- While value in *high* is larger than the pivot, move the pointer *high* to the left

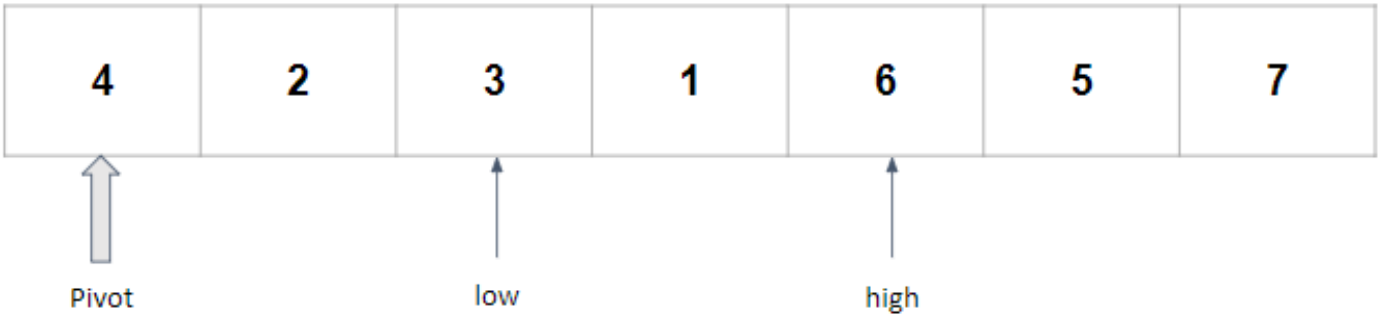


Quicksort Algorithm

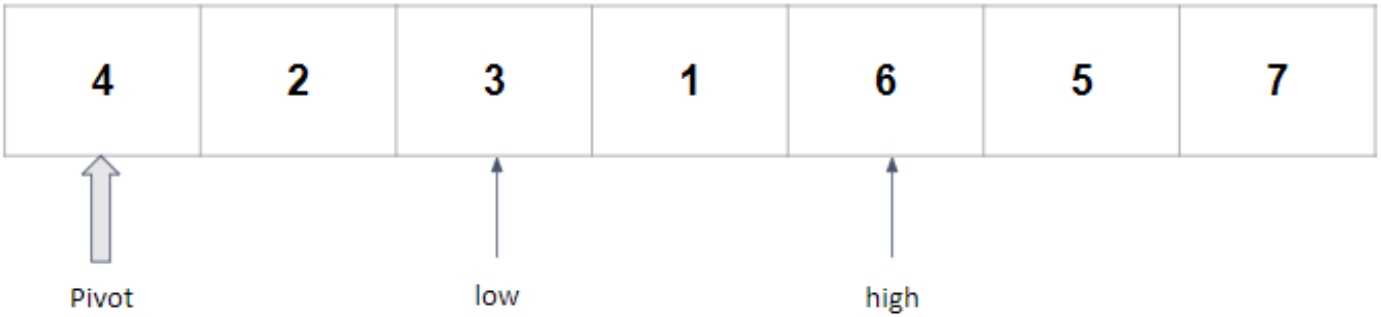
- When *low* has a value larger than the pivot and *high* has a value smaller than the pivot, then exchange the values in *low* and *high*



Quicksort Algorithm



Quicksort Algorithm

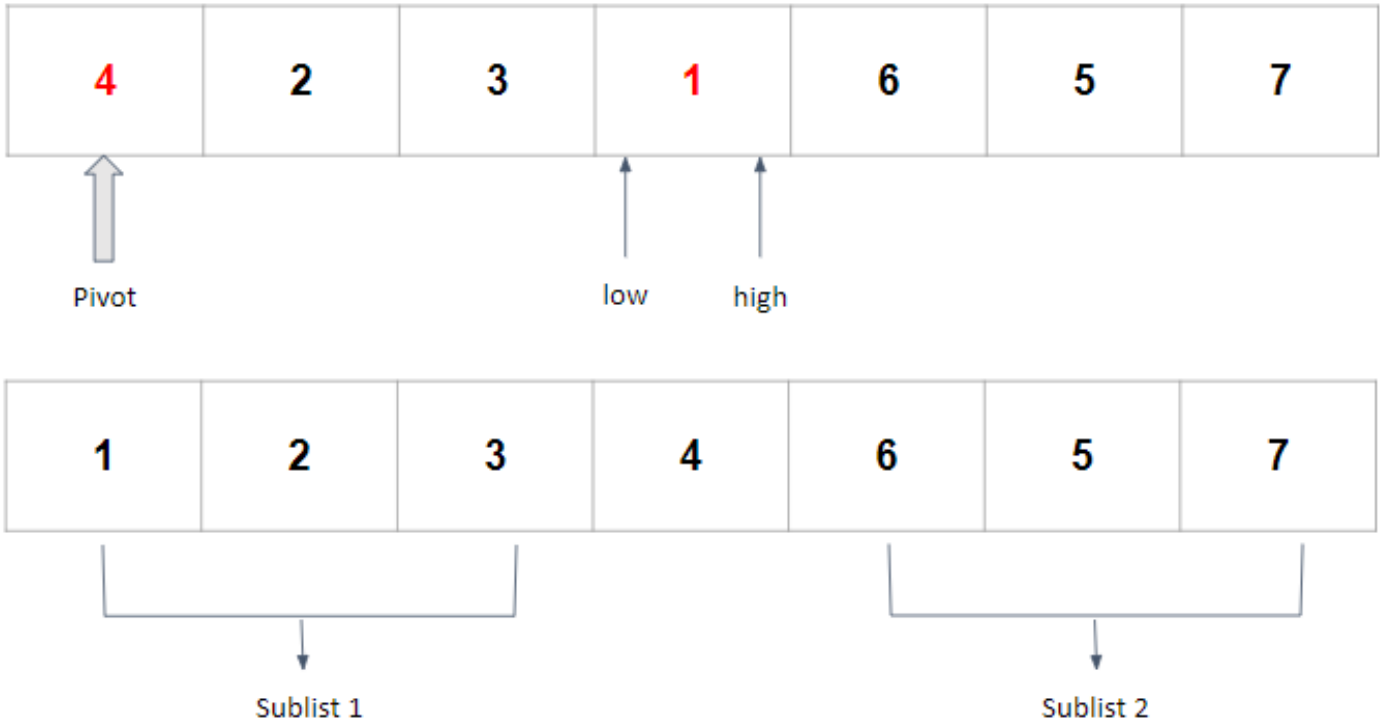


Quicksort Algorithm

- Finally, when *low* $\geq$ *high*, let the index of the element at which they crossed over be i. Exchange element at i and the pivot

Quicksort Algorithm

- Finally, when *low* $\geq$ *high*, let the index of the element at which they crossed over be i. Exchange element at i and the pivot



- Recursively, sort the two sublists

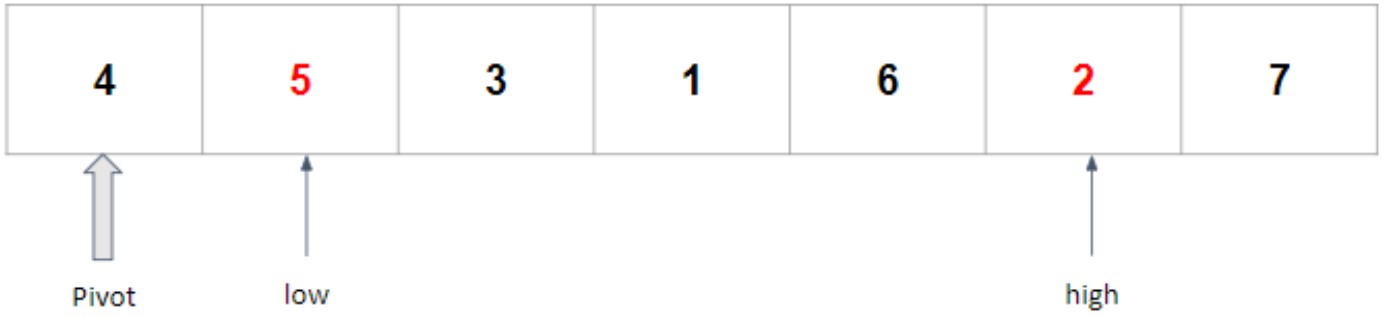
Quicksort Algorithm

```
quicksort(A,low,high){
    if low < high
        s <- partition(A,low,high)
        quicksort(A,low,s-1)
        quicksort(A,s+1,high)
}
```

Partition Algorithm

```
partition(A,lw,hg){
    p <- A[lw], low <- lw, high <- hg+1

    while low < high
        while  A[low] < p
            low <- low + 1
        while  A[high] > p
            right <- high - 1
        swap(A[low],A[high])
    swap(A[low],A[high])
    return high
}
```



Quicksort Analysis - Time complexity

- Choice of pivot decides the time complexity
- An ideal scenario is to pick the median of values as the pivot
- A naive algorithm to find the median needs to sort the list and pick the middle element: this is cyclic - sort the list to sort the list
- If the pivot splits the list, ideally, into two equal halves, then the recurrence is:

Quicksort Analysis - Time complexity

- Choice of pivot decides the time complexity
- An ideal scenario is to pick the median of values as the pivot
- A naive algorithm to find the median needs to sort the list and pick the middle element: this is cyclic - sort the list to sort the list
- If the pivot splits the list, ideally, into two equal halves, then the recurrence is:

$$T(n) = 2T(n/2) + c.n$$

- In this case, the time complexity is $O(n \log n)$ (same as mergesort)

Quicksort Analysis - Time complexity

- In the worst case, if the pivot splits the list into two sublists, where one list is of length 0 and the other is of length $n - 1$, then the recurrence is:

Quicksort Analysis - Time complexity

- In the worst case, if the pivot splits the list into two sublists, where one list is of length 0 and the other is of length $n-1$, then the recurrence is:

$$T(n) = T(n-1) + T(0) + n$$

and the complexity can be computed as

$$\begin{aligned} T(n) &= T(n-1) + n \\ &\dots \\ &= O(n^2) \end{aligned}$$

Quicksort Analysis - Time complexity

- Does this mean that the quicksort is worse than mergesort?

Quicksort Analysis - Time complexity

- Does this mean that the quicksort is worse than mergesort?
- Not necessarily. An average case analysis shows that the quicksort takes $O(n \log n)$
- With random data, there will be a good mix of balancing that happens at various levels of splits
- Instead of relying on random data, we could also look at a random pivot, which also leads to an average complexity of $O(n \log n)$

Quicksort Analysis - Space complexity

- No additional space as was needed for mergesort

Mergesort vs Quicksort

Mergesort

```
mergesort(...)
mergesort(...)
merge(...)
```

Quicksort

```
partition(...)
quicksort(...)
quicksort(...)
```

Summary

- Uses divide and conquer strategy
- Finds an elements, namely the pivot, to partition the input into two sublists such that one sublist has values smaller than the value of the pivot, and the other list has values larger than that of the pivot
- Recusively sort the two sublists
- Time complexity: Average case time complexity $O(n \log n)$, Worst case time complexity $O(n^2)$
- Space complexity: No additional space other than the stack space for recursion (which can be $O(\log n)$ or $O(n)$ based on the input)
- In-place
- Not stable